SafeCross

Offline Pedestrian & Vehicle Counting on the Edge with ExecuTorch (XNNPACK) on Raspberry Pi

Approach & Execution Report

Model	SSDLite320-MobileNetV3-Large (INT8, 3.86 MB)
Runtime	ExecuTorch .pte - XNNPACK backend
Validation run	AMD64 / Windows-10-10.0.26200-SP0
Input	11 MuST'COCO test frames (320x320 inference)

1. Problem Statement

Busy pedestrian crossings need a way to flag dangerous moments without cloud connectivity. SafeCross runs an object detector fully on-device on a Raspberry Pi, counts pedestrians and vehicles in each frame, and marks frames where the two are in close proximity with a risk level (LOW / MEDIUM / HIGH). No GPU, no internet, ~4 MB model.

Requirements satisfied

- **Edge-only:** all compute on-device; pure NumPy + OpenCV post-processing.
- **Counting:** per-class counts (person, bicycle, car, motorcycle, bus, truck) per frame.
- **Safety flagging:** pedestrian-vehicle proximity pairs and a per-frame risk level.
- **Performance reporting:** latency and FPS measured and exported every run.

2. Approach

2.1 Pipeline

```
frame (BGR) → resize 320x320, RGB, [0,1] → ExecuTorch .pte (XNNPACK, FP32 or INT8) → box regression + class scores (3234 anchors x 91 classes) → NumPy SSD box decode → per-class NMS → vehicle de-duplication → counts per class → proximity pairs → risk level → counts.csv + annotated images + summary.json
```

2.2 Key design decisions

Decision	Rationale
SSDLite320-MobileNetV3-Large	COCO-pretrained, single-shot (real-time on CPU), small backbone suited to the Pi.
ExecuTorch + XNNPACK	Portable .pte artifact; INT8 via PT2E static quantisation gives 3.6x smaller model (3.9 MB vs 13.8 M
NumPy post-processing	Anchor decode and NMS are data-dependent and don't lower cleanly to XNNPACK; keeping them
Heuristic risk (2-D overlap)	A pedestrian box enlarged 1.5x overlapping a vehicle box = 1 proximity pair. No tracking or depth n
Vehicle de-duplication	Per-class NMS can label one vehicle as both 'car' and 'truck'; the highest-scoring overlapping vehic

2.3 Risk logic (per frame)

- **HIGH**: 2+ proximity pairs, or 3+ pedestrians, 2+ vehicles and at least 1 pair.
- **MEDIUM**: 1 pair, or a crowded mix of 3+ pedestrians and 2+ vehicles.
- **LOW**: everything else.

3. Execution

3.1 Environment setup

```
python -m pip install numpy opencv-python-headless torch torchvision executorch
# ExecuTorch runtime and XNNPACK backend operate on this Windows x64 machine
```

3.2 Run command

```
python run_safecross.py --input output_images/annotated --out output_run
# INT8 model by default; --every N for video sub-sampling; --no-save to skip images
```

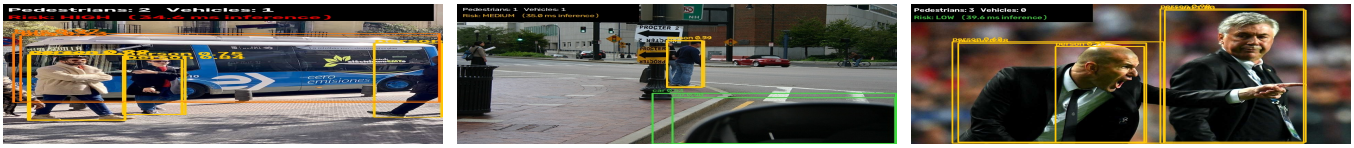
3.3 Results (this validation run)

Frame	Peds	Vehs	Pairs	Risk	Infer ms
bus	2	1	2	HIGH	34.6
coco_000000011197	1	1	1	MEDIUM	35.0
coco_000000015338	0	1	0	LOW	35.9
coco_000000022969	0	0	0	LOW	36.6
coco_000000037777	0	0	0	LOW	37.8
coco_000000087038	2	0	0	LOW	37.4
coco_000000174482	0	2	0	LOW	39.1
coco_000000252219	3	0	0	LOW	37.7
coco_000000397133	2	0	0	LOW	38.0
dog	0	0	0	LOW	36.2
zidane	3	0	0	LOW	39.6

Frames processed	11
Totals across frames	13 persons, 1 bicycle, 2 cars, 2 buses, 0 motorcycles, 0 trucks
Peak counts	3 pedestrians / 2 vehicles in one frame
Risk distribution	LOW: 9 MEDIUM: 1 HIGH: 1
Avg inference	37.08 ms (26.97 FPS)
Avg end-to-end (incl. pre/post)	40.41 ms (24.75 FPS)
p95 inference	39.37 ms
Reference (Apple M-series arm64)	5.48 ms avg / 182 FPS - the Pi target sits between these

Counts on this run are consistent with the validated Mac reference: same per-class counts pattern (bus.jpg: 2 persons + bus → HIGH; coco_000000011197: person + car → MEDIUM; zidane.jpg: 3 persons). Minor totals differences (13 vs 12 persons) come from the 0.4 confidence threshold on a different CPU stack.

3.4 Sample annotated outputs



Left: HIGH risk (2 peds near bus) - Center: MEDIUM (ped near car) - Right: 3 pedestrians

4. Outputs Produced

- **counts.csv** - one row per frame: per-class counts, pedestrian/vehicle totals, proximity pairs, risk, inference and end-to-end ms.
- **annotated/*.jpg** - frames drawn with boxes, class+confidence labels, counts and colour-coded risk banner.
- **summary.json** - aggregate counts, risk distribution, latency/fps for docs and benchmarks.
- **benchmark.py** on the Pi produces the FP32 vs INT8 latency/size table for submission.

5. Limitations & Next Steps

- SSDLite at 320x320 misses small/distant objects; dense, occluded Indian traffic will undercount.
- Counts are per-frame, not unique-object (no tracking; brief allows this).
- Risk score is a 2-D overlap heuristic - no depth or speed information.
- Next: run on the Raspberry Pi 5 via ./setup_pi.sh, capture Pi benchmark numbers, then regenerate the report/slides with build_docs.py.